

CG_HW3_PoissonImageEditing 实验报告

李亭皓 PB23010374

2026 年 3 月 22 日

1 实验目标与整体思路

本次实验的核心任务是实现 Poisson Image Editing，我的实现分为两部分：一部分负责从用户选区得到离散区域，另一部分负责建立 Poisson 线性系统并求解。前者主要包括矩形选区与自由绘制选区，其中自由绘制区域的内部填充采用扫描线算法；后者则以基础 Poisson 为核心，并进一步实现了 Mixed Gradient、Color Constraint 和 Alpha Matte 三种改进方式。

在实验中，我尤其关注**色彩渗透**问题。基础 Poisson 的边界过渡通常较自然，但当目标背景与主体颜色差异较大时，边界失配会通过调和插值扩散到整个区域内部，使主体原有颜色被背景“染色”。

2 基础 Poisson 融合实现

2.1 区域表示与未知量编号

我用 PoissonRegion 表示融合区域。对于 mask 中每个非零像素 (s_x, s_y) ，根据源图锚点和目标图锚点之间的位移，将其映射到目标图位置 (t_x, t_y) ：

$$(t_x, t_y) = (a_t^x + s_x - a_s^x, a_t^y + s_y - a_s^y). \quad (1)$$

每个区域像素都对应一个 RegionPixel，记录其 source 坐标、target 坐标和编号 id 。这样，区域内共有 n 个像素时，就得到 n 个未知量。

随后建立 source 坐标到未知量编号的反查表，便于快速判断某个邻居是否仍在区域内部。

2.2 边界信息与离散矩阵构造

对于区域内每个像素 p ，检查其四邻域。若某个邻居 q 不在区域内，则当前像素为边界像素，并记录该邻居在 source 和 target 中的坐标。这样可以得到边界像素集合以及每个边界像素对应的区域外邻居列表。

在此基础上构造四邻域离散 Laplace 矩阵 A 。对区域内像素 p ，离散方程写为

$$4f_p - \sum_{q \in N(p) \cap \Omega} f_q = b_p. \quad (2)$$

因此矩阵 A 的规则为：

- 对角元 $A_{pp} = 4$;
- 若邻居 q 也在区域内，则 $A_{pq} = -1$;
- 若邻居 q 在区域外，则不写入矩阵，而在右端项中单独处理。

2.3 基础 Poisson 模型

基础 Poisson 融合通过保持源图梯度并匹配目标边界来求解结果图像 f ：

$$\min_f \int_{\Omega} \|\nabla f - \nabla g\|^2 d\Omega, \quad f|_{\partial\Omega} = f^*|_{\partial\Omega}, \quad (3)$$

其中 g 为 source 图像， f^* 为 target 图像。其离散后写成

$$Af = b. \quad (4)$$

在实现中，我先收集区域内部 source 像素值构成向量 g_{in} ，并先计算

$$b = Ag_{\text{in}}. \quad (5)$$

这对应 importing gradients 的散度项。然后，对于边界像素 p 及其区域外邻居 q ，补充边界修正：

$$b_p \leftarrow b_p - g_q + f_q^*. \quad (6)$$

其中 g_q 用于补回被截断的源图梯度项， f_q^* 则施加目标图的 Dirichlet 边界条件。

2.4 求解器与矩阵复用

我将求解器部分封装在 `PoissonSolver` 中。其 `build()` 先建立系统矩阵，再使用稀疏 LDLT 分解：

$$A = LDL^T. \quad (7)$$

对 RGB 三个通道，只需要分别构造不同的右端项 $b^{(c)}$ ，再调用同一个分解后的求解器求得

$$Ax^{(c)} = b^{(c)}, \quad c \in \{R, G, B\}. \quad (8)$$

最后将三个通道结果写回 target 图像。

2.5 clamp 处理方式

这里我的实现与文档不同，希望可以复用 A 。

具体来说，当某个边界外邻居映射到 `target` 后超出画布边界时，我并没有重建矩阵 A ，而只是把读取 `target` 边界值时的坐标做限制：

$$\hat{t}_x = \text{clamp}(t_x, 0, W - 1), \quad \hat{t}_y = \text{clamp}(t_y, 0, H - 1). \quad (9)$$

然后使用 $(\hat{t}_x, \hat{t}_y)$ 处的 `target` 像素作为边界项。

这样做的目的，是让区域保持不变，从而继续复用同一个 A 及其预分解结果。也就是说，拖动过程中主要变化的是右端项 b ，而不是整个线性系统结构。这种实现更符合我对实时交互效率的要求。

3 扫描线算法实现

自由绘制选区采用经典的扫描线算法。

对于每一个整数纵坐标 y ，扫描线算法会计算水平线与多边形边界的所有交点。若一条边的两个端点分别为 (x_1, y_1) 和 (x_2, y_2) ，则在扫描线 y 上的交点横坐标为

$$x = x_1 + \frac{(y - y_1)(x_2 - x_1)}{y_2 - y_1}. \quad (10)$$

将所有交点按横坐标排序后，两两配对为区间 $[x_{2k}, x_{2k+1}]$ ，再把区间内所有整数像素加入内部像素集合。

实现时采用半开区间规则 $y \geq y_1, y < y_2$ ，并忽略水平边，以避免扫描线恰好经过顶点时的重复计数问题。最终，扫描线算法输出的是一组内部像素坐标，这些坐标被写入 `mask`，再交给 `PoissonRegion` 统一处理。

因此，扫描线部分负责“从轮廓得到离散区域”，`Poisson` 部分负责“对该离散区域建立线性系统并求解”，两者职责清晰分离。

4 Poisson 改进方法

4.1 改进动机：色彩渗透

基础 `Poisson` 的主要优点是边界自然，但也存在一个明显问题：它本质上是在区域内部构造一个调和插值场，将边界失配扩散到整个区域内部。若 `source` 主体与 `target` 背景在边界附近颜色差异较大，则这种扩散往往会导致主体整体颜色发生明显偏移，也就是色彩渗透。

针对这一问题，我实现了三种改进方式，分别从指导梯度、能量函数以及修正范围三个角度进行调整。

4.2 1. Mixed Gradient: 保留更强纹理

Mixed Gradient 的思想是：对每条四邻域边 (p, q) ，不再固定使用 source 的梯度，而是比较 source 和 target 上该边的差分，选取绝对值更大的一个作为指导场：

$$v_{pq} = \begin{cases} g_p - g_q, & |g_p - g_q| \geq |f_p^* - f_q^*|, \\ f_p^* - f_q^*, & |g_p - g_q| < |f_p^* - f_q^*|. \end{cases} \quad (11)$$

于是离散方程变为

$$4f_p - \sum_{q \in N(p) \cap \Omega} f_q = \sum_{q \in N(p)} v_{pq} + \sum_{q \in N(p) \setminus \Omega} f_q^*. \quad (12)$$

这一方法的关键在于**修改右端项中的指导场**，而不是修改矩阵。代码中，对每个像素 p 和四邻居 q ，分别计算

$$g_s = g_p - g_q, \quad g_t = f_p^* - f_q^*,$$

再取绝对值较大的一个加入右端项。若 q 不在区域内，则再补充对应的 target 边界值。

这种方法的优点是纹理保护更强，尤其适合 source 和 target 都有较明显局部结构时的情况。但它需要逐边同时访问 source 和 target，因此右端项构造更复杂，速度相对较慢；此外，当两幅图的纹理差异过大时，结果有时会显得不够协调。

4.3 2. Color Constraint: 通过能量项显式保色

为了更直接地抑制色彩渗透，我实现了带颜色约束的 Poisson 变体。其能量函数在基础 Poisson 项之外，再增加一个只在主体区域内生效的颜色保真项：

$$E(f) = \sum_{(p,q) \in \mathcal{E}} [(f_p - f_q) - (g_p - g_q)]^2 + \lambda \sum_{p \in \Omega} m_p (f_p - g_p)^2. \quad (13)$$

这里 $m_p \in \{0, 1\}$ 是主体指示函数，表示像素 p 是否属于 object of interest (OOI)。

对该能量求极值，可得到

$$(A + \lambda D)f = b_{\text{pois}} + \lambda Dg, \quad (14)$$

其中 $D = \text{diag}(m_p)$ 为对角矩阵。可以看到，这种方法的本质是：

- 左边对系统矩阵增加对角项 λD ；
- 右边增加保色项 λDg 。

在实现上，我直接使用 `subject_mask` 构造 OOI。对于区域内像素 p ，若 `subject_mask(p)>0`，则令

$$m_p = 1, \quad (15)$$

否则 $m_p = 0$ 。因此，这里的 OOI 就是 `subject_mask` 所指示的主体区域。

这种方法能明显增强主体颜色保持能力，但也引入了参数 λ 。它需要结合具体图像调节参数，目前在实现的实现中仍然比较容易出现边界不够自然的情况。

4.4 3. Alpha Matte: 缩小 Poisson 修正的作用范围

第三种方法参考了 Ding 和 Tong 的论文 *Content-aware copying and pasting in images*。论文的核心思想是：基础 Poisson 的问题在于边界颜色差会被无差别地扩散到整个区域内部，因此可以引入一个内容相关的权重函数来控制这种扩散。论文将原始泛函修改为

$$J(u) = \iint_{\Omega} \left\| \nabla \left(\frac{u - g}{\omega} \right) \right\|^2 d\Omega, \quad u|_{\partial\Omega} = f^*|_{\partial\Omega}, \quad (16)$$

并令

$$\tilde{u} = \frac{u - g}{\omega}, \quad u = \omega \tilde{u} + g. \quad (17)$$

论文进一步使用 alpha matte 构造权重函数，并取

$$\omega = 1 - \alpha. \quad (18)$$

我的实现没有直接复现论文中的 MVC 求解，而是保留了 Poisson 线性系统框架，只借鉴其 “ $\omega = 1 - \alpha$ 控制修正范围” 的思想。具体过程如下：

1. 根据 `subject_mask` 构造二值图；
2. 使用 `AlphaMaskBuilder` 对该二值图做两次 box blur，得到平滑的 α ；
3. 对区域内像素直接取

$$\alpha_p = \text{blurredMask}(p), \quad \omega_p = 1 - \alpha_p; \quad (19)$$

4. 只利用 boundary 外邻居的 target-source 差值构造修正场：

$$Au = b, \quad b_p = \sum_{q \in N(p) \setminus \Omega} (f_q^* - g_q); \quad (20)$$

5. 最终输出为

$$f_p = g_p + \omega_p u_p. \quad (21)$$

这里的含义是：在主体内部， α 较大，因此 $\omega = 1 - \alpha$ 较小，Poisson 修正作用较弱，主体原有颜色得以保留；在边界过渡带， α 下降、 ω 增大，因此修正项主要集中作用于边界附近，使过渡更自然。

与 Color Constraint 相比，Alpha Matte 不是硬性地把结果拉回 source 颜色，而是通过软权重控制修正范围；与 Mixed Gradient 相比，它不需要逐边比较双图梯度，因此整体更稳定，也更快。

4.5 Mask 的具体取法

三种方法都依赖用户给出的选区，但作用方式不同：

- **基础 Poisson / Mixed Gradient**：只依赖选区 `mask`，用于定义融合区域 Ω ；
- **Color Constraint**：额外使用 `subject_mask`，其非零像素直接定义为 OOI，用于构造 D ；
- **Alpha Matte**：同样使用 `subject_mask`，但先将其模糊为软 `mask`，再取 $\omega = 1 - \alpha$ 。

因此，Color Constraint 和 Alpha Matte 都依赖主体 `mask`，但前者是**硬约束保色**，后者是**软权重限制修正范围**。

5 实验结果与分析

图 1 展示了我的实验结果，其中 (a) 为直接粘贴选区示意，(b) 为基础 Poisson，(c) 为 Mixed Gradient，(d) 为 Color Constraint，(e) 为 Alpha Matte。



(a) 选区与粘贴位置 (b) Basic Poisson (c) Mixed Gradient (d) Color Constraint (e) Alpha Matte

图 1: 基础方法与三种改进方法的实验结果对比

从结果上看，基础 Poisson 的边界过渡自然，但主体颜色明显受背景影响，色彩渗透较为严重。Mixed Gradient 对纹理保护较好，尤其能更好地保留局部细节，但求解速度相对较慢，而且当 `source` 与 `target` 的纹理差异过大时，画面可能显得不够理想。Color Constraint 能较直接地保持主体原有色彩，但对参数 λ 比较敏感，目前较容易在边界处产生不够自然的过渡。

相比之下，Alpha Matte 在我的实验中综合表现最好：求解较快，主体色彩保持较好，同时边界过渡仍然自然，更符合我“抑制色彩渗透”的目标。因此，在当前实现和实验设置下，我认为 Alpha Matte 是最适合这一任务的改进方式。

6 总结

本次实验中，我首先实现了基础 Poisson 融合，并通过扫描线算法支持自由绘制选区。随后，我针对色彩渗透问题实现了三种改进方案：

- Mixed Gradient：通过修改指导梯度场保留更强纹理；
- Color Constraint：通过修改能量函数，在主体区域显式保留颜色；

- Alpha Matte: 通过软权重缩小 Poisson 修正的作用范围, 在主体保色和边界自然之间取得更好平衡。

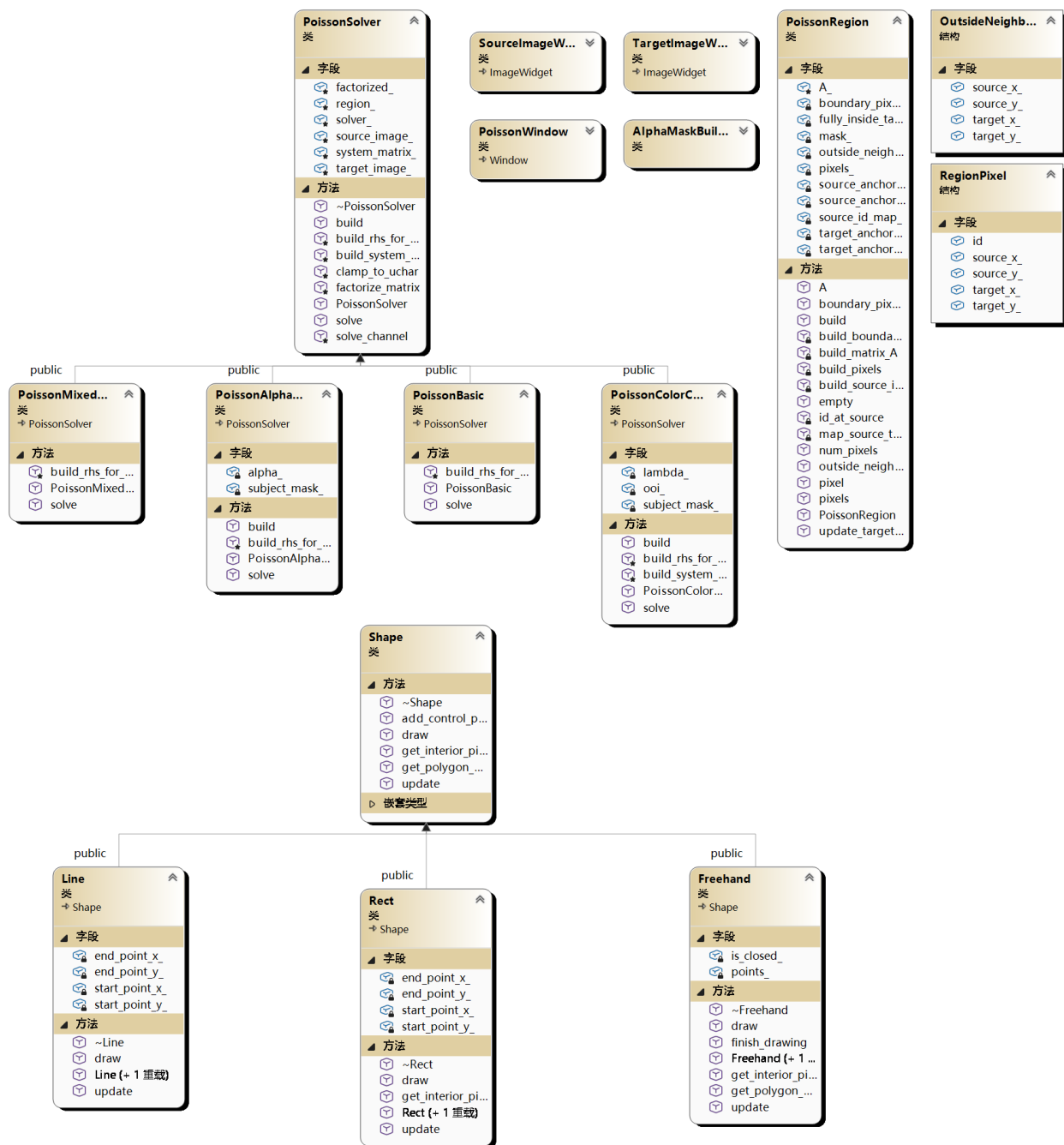


图 2: Poisson 图像编辑实现的主要类图

参考文献

- [1] Meng Ding, Ruofeng Tong. Content-aware copying and pasting in images. *The Visual Computer*, 26(6-8):721-729, 2010.